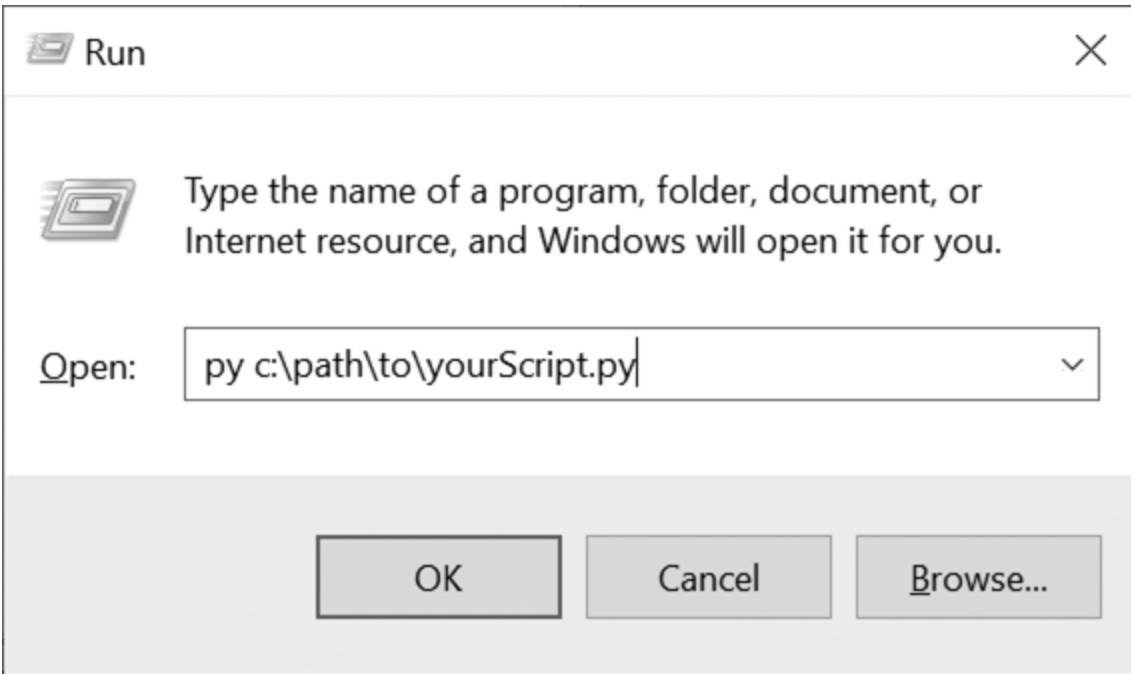




[Figure 2-7:](#) The Run dialog on Windows

Still, this method requires you to enter your script's full path. Also, the terminal window that displays the program's output will automatically close when the program ends, and you might miss some output.

You can solve these problems by creating a *batch script*, which is a small text file with the *.bat* file extension that can run multiple terminal commands at once, much like a shell script in macOS and Linux. You can use a text editor, such as Notepad, to create these files. Make a new text file containing the following two lines:

---

```
@py.exe c:\path\to\yourScript.py %*
@pause
```

---

Replace this path with the absolute path to your program, and save this file with a *.bat* file extension (for example, *yourScript.bat*). The @ sign at the start of each command prevents it from being displayed in the terminal window, and the %\* forwards any command line arguments entered after the batch filename to the Python script. The Python script, in turn, reads the command line arguments in the `sys.argv` list. This batch file will spare you from having to type the Python program's full absolute path every time you want to run it. The `@pause` command adds `Press any key to continue...` to the